ALA :3

NAME: Parmar Vishva j

EN NO: 240905091015

SUB: CRYPTOGRAPHY AND NETWORK SECURITY

CODE: BETIT16326

TOPIC: MAC-Based Message Authentication

HMAC (Hash-based Message Authentication Code) using SHA-256

1. Theory

HMAC (Hash-based Message Authentication Code) is a cryptographic technique used to verify the integrity and authenticity of a message. It uses a secret key along with a hash function like SHA-256 to generate a unique code (MAC). Only parties who know the secret key can generate or verify this MAC.

2. Steps Explanation

Step 1: Import required libraries (hmac and hashlib).

Step 2: Define a shared secret key between sender and receiver.

Step 3: Sender converts message into bytes and generates MAC using HMAC-SHA256.

Step 4: Sender sends message and MAC to receiver.

Step 5: Receiver recalculates MAC using same key.

Step 6: Receiver compares received MAC with calculated MAC.

Step 7: If MAC matches → message is authentic, else → message is tampered.

3. Python Code

```
import hmac
import hashlib

# THE SHARED SECRET KEY (Must be known by both parties)
SECRET_KEY = b'gmiu_secret_key_2026'

def sender_side(message_text):
    message_bytes = message_text.encode('utf-8')

    # Generate MAC using HMAC-SHA256
    mac = hmac.new(SECRET_KEY, message_bytes, hashlib.sha256).hexdigest()

    print("--- SENDER ---")
    print(f"Message: {message_text}")
    print(f"Generated MAC: {mac}")
```

```
    return message_text, mac

def receiver_side(received_message, received_mac):
    message_bytes = received_message.encode('utf-8')

    # Re-calculate MAC using the shared key
    expected_mac = hmac.new(SECRET_KEY, message_bytes, hashlib.sha256).hexdigest()

    print("\n--- RECEIVER ---")
    print(f"Expected MAC: {expected_mac}")

    # Compare MACs
    if hmac.compare_digest(received_mac, expected_mac):
        print("Verification Successful: Message is authentic and intact.")
    else:
        print("Verification Failed: Message or Key is invalid!")

# Simulation
msg, tag = sender_side("Transfer 5000 INR to vishva parmar")
receiver_side(msg, tag)

print("\n" + "!" * 40)
print("SIMULATING TAMPERING...")
tampered_msg = "Transfer 50000 INR to vishva parmar"
receiver_side(tampered_msg, tag)
```

output

1. Normal Transmission

```
--- SENDER ---
Message: Transfer 5000 INR to vishva parmar
Generated MAC: <some long hash value>

--- RECEIVER ---
Expected MAC: <same long hash value>
Verification Successful: Message is authentic and intact.
```

2. Tampered Transmission

```
SIMULATING TAMPERING...

--- RECEIVER ---
Expected MAC: <different hash value>
Verification Failed: Message or Key is invalid!
```

4. Code Explanation

The sender generates a MAC using a secret key and SHA-256 algorithm. The receiver recalculates the MAC using the same key and compares it with the received MAC. If both match, the message is authentic and unchanged. If not, it indicates tampering or incorrect key.

5. Conclusion

HMAC ensures both message integrity and authentication. It is widely used in secure communication systems.